EXPERIMENT 2 — INTER-PROCESS COMMUNICATION (IPC)

AIM

To study and implement various **Inter-Process Communication (IPC)** mechanisms, including **direct messaging**, **message queues**, and **publish/subscribe models**, for enabling efficient data exchange and coordination between processes in both local and distributed environments.

Learning Objective

To understand and implement Inter-Process Communication, message-based communication, and the Publish/Subscribe model to enable processes and components to exchange data efficiently and asynchronously.

Learning Context

This experiment is situated within the broader study of how operating systems and distributed applications enable communication and coordination among independent processes. As modern software systems increasingly rely on parallel execution, microservices, and event-driven architectures, understanding fundamental communication mechanisms becomes essential. Through Inter-Process Communication (IPC), message queues, and the Publish/Subscribe model, students explore how data flows between processes in both local and distributed environments. The lab also introduces real-world messaging technologies such as RabbitMQ, which implements advanced queuing and pub-sub patterns used in cloud platforms, enterprise integration, IoT systems, and large-scale distributed applications. This context helps students connect basic IPC concepts with practical, industry-grade messaging solutions that support scalability, reliability, and asynchronous communication.

Introduction

- A process is an independent program under execution. Each process has its own Memory space, Variables, Files, Stack, CPU state. By default, processes cannot access each other's memory or data, because modern operating systems provide memory protection and isolation for safety.
- Inter-Process Communication (IPC) refers to the methods and mechanisms that allow two or more processes to Exchange data, Coordinate their activities, Synchronize execution.
- IPC is essential because real-world applications rarely run as a single process. Operating systems, servers, browsers, databases, and cloud applications are all composed of many processes that need to collaborate.
- Processes communicate to share data, for coordinating tasks, improving performance, building distributed systems.
- IPC mechanisms fall into two broad categories:
    - Shared Memory:
        - Multiple processes access the same memory region.
        - Fastest method (no copying of data).

        - Requires synchronization (mutex/locks) to avoid conflicts.

    Use cases: High-speed trading, games, databases.

- Message Passing:
    - Processes send and receive messages through OS or a messaging system.
    - Safer (no shared memory).
    - Easier to use but may be slower due to copying.

    - Two kinds:

    1. Direct Messaging (Local IPC)
        - Provided by the OS:
        - Pipes
        - Named Pipes (FIFOs)
        - Message Queues
        - Signals
        - Semaphores
        - Sockets

    2. Network-based Message Systems (Distributed IPC)
        - Used across machines:
        - ZeroMQ
        - RabbitMQ
        - Kafka
        - gRPC
        - REST APIs

EXERCISE 2.A — IPC USING PYTHON MULTIPROCESSING PIPE

AIM
To implement Inter-Process Communication between a parent process and a child process using Python's multiprocessing Pipe, demonstrating direct, two-way message passing in a local IPC environment.

SOFTWARE / TOOLS USED
- Python 3.x installed
- A code editor (VS Code / PyCharm / Notepad++ / etc.)
- Terminal / Command Prompt

Procedure
STEP 1 — Create a pipe

parent_conn, child_conn = Pipe()

- parent_conn is used by the parent process
- child_conn is used by the child process
- Both ends can send() and recv()

Step 2 — Start a child process

The child will run a function that receives and sends data using child_conn.

Step 3 — Parent sends a message

parent_conn.send("Hello Child")

Step 4 — Child receives it

msg = child_conn.recv()

Step 5 — Child sends a reply

child_conn.send("Hello Parent")

Step 6 — Parent receives reply

reply = parent_conn.recv()

Python Program:

```
from multiprocessing import Process, Pipe

def child_process(conn):
    print("[Child] Waiting for message...")
```

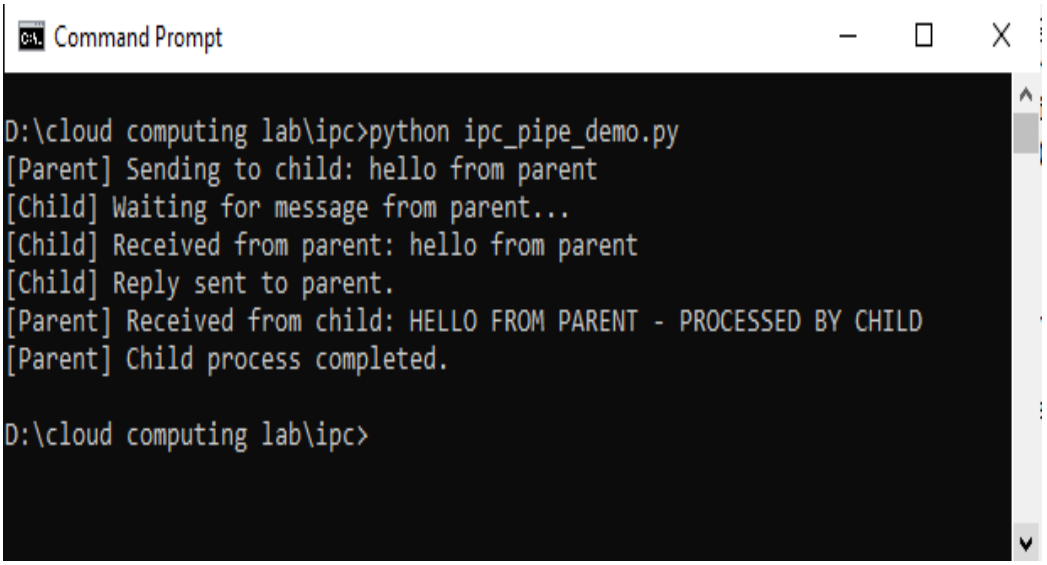
```
    # receive from parent
    msg = conn.recv()
    print("[Child] Received:", msg)

    # send reply
    reply = msg.upper() + " (from child)"
    conn.send(reply)
if __name__ == "__main__":
    parent_conn, child_conn = Pipe()

    # start child process
    p = Process(target=child_process, args=(child_conn,))
    p.start()
    # parent sends a message
    parent_conn.send("hello child")
    print("[Parent] Sent: hello child")

    # parent receives reply
    print("[Parent] Received:", parent_conn.recv())
    p.join()
```

Execution

D:\cloud computing lab\ipc>python ipc_pipe_demo.py
[Parent] Sending to child: hello from parent
[Child] Waiting for message from parent...
[Child] Received from parent: hello from parent
[Child] Reply sent to parent.
[Parent] Received from child: HELLO FROM PARENT - PROCESSED BY CHILD
[Parent] Child process completed.

Output Screens

Screenshot 1: CMD showing executing ipc_pipe_demo.py



EXERCISE 2.B — IPC USING PYTHON MULTIPROCESSING QUEUE

AIM

To demonstrate Inter-Process Communication using Python's multiprocessing Queue, enabling safe, synchronized message exchange among multiple processes using the FIFO (First-In-First-Out) mechanism.

IPC Using Python Multiprocessing Queue *Direct Messaging (Local IPC)*

- A Queue is an IPC mechanism that allows multiple processes to safely send and receive messages.
- It behaves like a real waiting line:
  - First message in → First message out (FIFO)
  - Many processes can put data in
  - Many processes can read data out
- Python's *multiprocessing.Queue* is thread-safe and process-safe, meaning it handles synchronization automatically.
- Use Queue when
  - more than two processes need to communicate
  - you have multiple workers and one manager
  - tasks/jobs/results need to be passed around
  - processes should not directly contact each other

SOFTWARE / TOOLS USED

- Python 3.x installed
- A code editor (VS Code / PyCharm / Notepad++ / etc.)
- Terminal / Command Prompt

PROCEDURE

STEP 1 — Create a queue

```
from multiprocessing import Queue
q = Queue()
```

STEP 2 — One process puts data into queue

```
q.put("Task 1")
```

STEP 3 — Another process reads from queue

```
data = q.get()
```

Python Program:

```
from multiprocessing import Process, Queue
import time
def worker(worker_id, task_queue):
    while True:
        task = task_queue.get()     # read task
        if task is None:            # stop signal
```

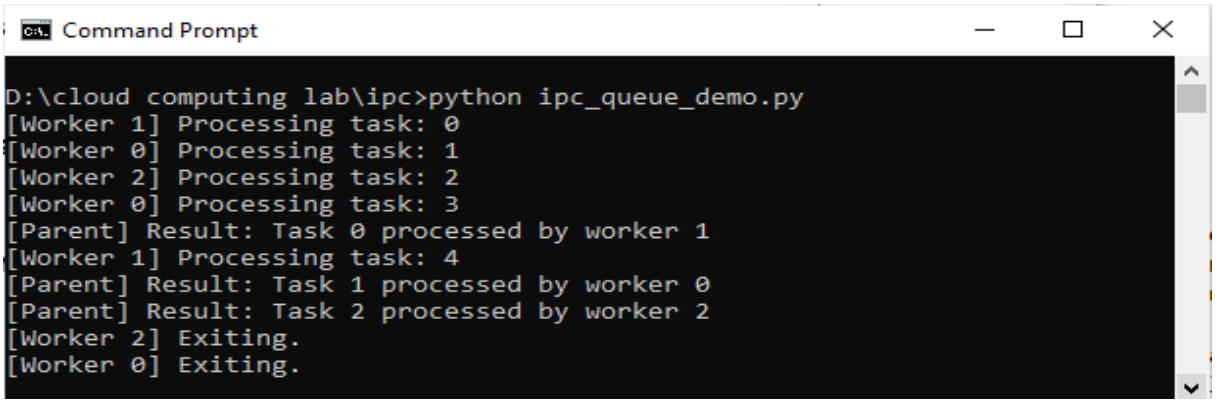
```
            print(f"Worker {worker_id} stopping.")
            break
        print(f"Worker {worker_id} got:", task)
        time.sleep(1)  # simulate work
if __name__ == "__main__":
    q = Queue()
    # Start 2 worker processes
    workers = []
    for i in range(2):
        p = Process(target=worker, args=(i, q))
        p.start()
        workers.append(p)
    # Add tasks
    for t in range(5):
        print("[Parent] Sending task", t)
        q.put(t)
    # Send stop signal to each worker
    for _ in workers:
        q.put(None)
    for p in workers:
        p.join()
    print("All workers finished.")
```

Execution

```
D:\cloud computing lab\ipc>python ipc_queue_demo.py
[Worker 1] Processing task: 0
[Worker 0] Processing task: 1
[Worker 2] Processing task: 2
[Worker 0] Processing task: 3
[Parent] Result: Task 0 processed by worker 1
[Worker 1] Processing task: 4
[Parent] Result: Task 1 processed by worker 0
[Parent] Result: Task 2 processed by worker 2
[Worker 2] Exiting.
[Worker 0] Exiting.
[Parent] Result: Task 4 processed by worker 1
[Worker 1] Exiting.
[Parent] Result: Task 3 processed by worker 0
[Parent] All workers finished.

D:\cloud computing lab\ipc>
```

Output Screens

Screenshot 1 — CMD showing executing ipc_queue_demo.py



VIVA QUESTIONS on IPC Pipes & Queues (Exercise 2.A & 2.B)

1. What is Inter-Process Communication (IPC)?
2. Why do processes need IPC?
3. What is the difference between a process and a thread?
4. What are the two major categories of IPC?
5. What is a Pipe?
6. Is a Pipe unidirectional or bidirectional?
7. What is the difference between a Pipe and a Named Pipe (FIFO)?
8. How does Python's multiprocessing.Pipe() work?
9. What are the two ends of a pipe called?
10. Can a Pipe be used by more than two processes? Why or why not?
11. What is a Queue in multiprocessing?
12. How does a Queue handle multiple producers and consumers safely?
13. What is the FIFO property?
14. Why is Queue better than Pipe for task distribution?
15. What synchronization issues can occur in IPC?
16. What happens if a process calls recv() when the pipe is empty?
17. What is the role of OS kernel in IPC?
18. Compare shared memory and message passing.
19. Why do modern systems prefer message passing over shared memory?

EXERCISE 2.C — MESSAGE-ORIENTED COMMUNICATION (REQUEST/REPLY SIMULATION)

AIM

To implement a message-oriented request–reply communication model using Python multiprocessing Queues, simulating client–server interaction through asynchronous message exchange.

Python Program: message_queue_client_server.py

```
from multiprocessing import Process, Queue
import time

def server(request_queue, response_queue):
    print("[Server] Started. Waiting for requests...")
    while True:
        request = request_queue.get()
        if request == "STOP":
            print("[Server] Stopping.")
            break
        client_id, payload = request
        print(f"[Server] Received from Client-{client_id}: {payload}")

        # Simulate some "business logic"
        time.sleep(1)
        response = payload[::-1]  # Reverse the string as a dummy operation

        response_queue.put((client_id, response))
        print(f"[Server] Sent response to Client-{client_id}")
def client(client_id, request_queue, response_queue, message):
    print(f"[Client-{client_id}] Sending: {message}")
    request_queue.put((client_id, message))
    # Wait for response
    while True:
        resp_client_id, resp = response_queue.get()
        if resp_client_id == client_id:
            print(f"[Client-{client_id}] Got response: {resp}")
            break
        else:
            # If response is for another client, put it back
            response_queue.put((resp_client_id, resp))
if __name__ == "__main__":
    request_queue = Queue()
```

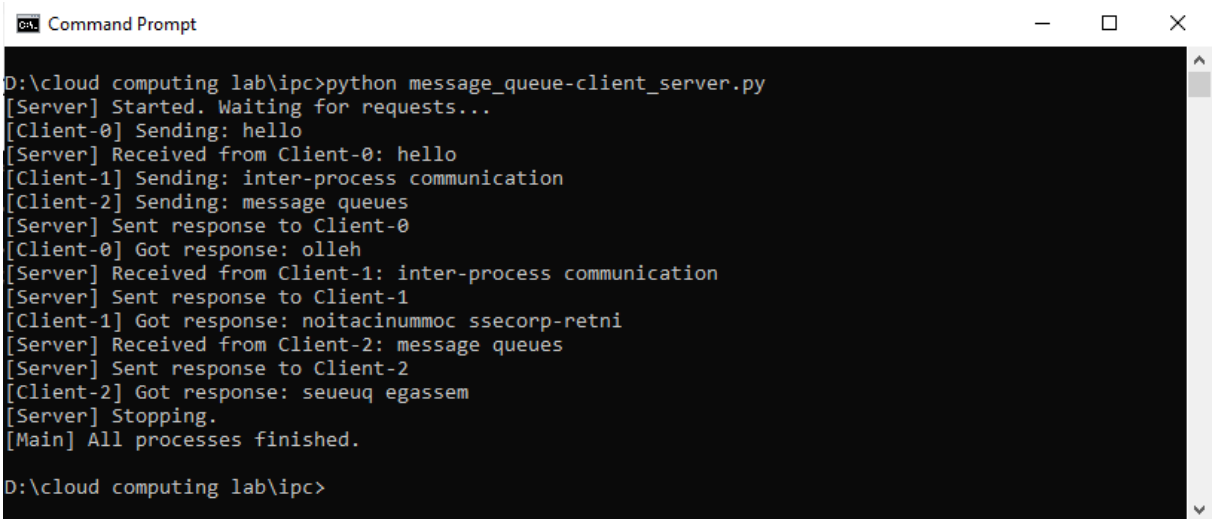
```
    response_queue = Queue()

    # Start server
    server_process = Process(target=server, args=(request_queue, response_queue))
    server_process.start()
    # Start clients
    messages = ["hello", "inter-process communication", "message queues"]
    clients = []
    for cid, msg in enumerate(messages):
        p = Process(target=client, args=(cid, request_queue, response_queue, msg))
        p.start()
        clients.append(p)
    # Wait for all clients
    for p in clients:
        p.join()
    # Tell server to stop
    request_queue.put("STOP")
    server_process.join()
    print("[Main] All processes finished.")
```

Execution

```
D:\cloud computing lab\ipc>python message_queue-client_server.py
[Server] Started. Waiting for requests...
[Client-0] Sending: hello
[Server] Received from Client-0: hello
[Client-1] Sending: inter-process communication
[Client-2] Sending: message queues
[Server] Sent response to Client-0
[Client-0] Got response: olleh
[Server] Received from Client-1: inter-process communication
[Server] Sent response to Client-1
[Client-1] Got response: noitacinummoc ssecorp-retni
[Server] Received from Client-2: message queues
[Server] Sent response to Client-2
[Client-2] Got response: seueuq egassem
[Server] Stopping.
[Main] All processes finished.
```

Output Screens

Screenshot 1 — CMD showing executing message_queue_client_server.py



```
Command Prompt

D:\cloud computing lab\ipc>python message_queue-client_server.py
[Server] Started. Waiting for requests...
[Client-0] Sending: hello
[Server] Received from Client-0: hello
[Client-1] Sending: inter-process communication
[Client-2] Sending: message queues
[Server] Sent response to Client-0
[Client-0] Got response: olleh
[Server] Received from Client-1: inter-process communication
[Server] Sent response to Client-1
[Client-1] Got response: noitacinummoc ssecorp-retni
[Server] Received from Client-2: message queues
[Server] Sent response to Client-2
[Client-2] Got response: seueuq egassem
[Server] Stopping.
[Main] All processes finished.

D:\cloud computing lab\ipc>
```

VIVA QUESTIONS

1. What is message-based communication?
2. Name two benefits of using message queues.
3. What is a producer? What is a consumer?
4. What is the role of a message queue in distributed systems?
5. Why are messages stored in a queue instead of directly sending to consumers?
6. What is load balancing in worker queues?
7. How does the consumer get messages from a queue?
8. What is a blocking get() call?
9. How does message ordering work in queues?
10. What happens if a consumer crashes after receiving a message?
11. What is the difference between synchronous and asynchronous messaging?
12. Why are queues preferred for parallel task processing?

EXERCISE 2.D — MESSAGE-ORIENTED COMMUNICATION (PUBLISH/ SUBSCRIBE SIMULATION)

AIM

To simulate the Publish/Subscribe communication model by implementing a broker-based messaging system using Python threads and queues, where publishers send messages on topics and subscribers receive messages based on their subscriptions.

Python Program: pubsub_simulation.py

```
import threading
import queue
import time
import random
class Broker:
    def __init__(self):
        # topic -> list of subscriber queues
        self.subscribers = {}
        self.lock = threading.Lock()
    def subscribe(self, topic, subscriber_queue):
        with self.lock:
            if topic not in self.subscribers:
                self.subscribers[topic] = []
            self.subscribers[topic].append(subscriber_queue)
            print(f"[Broker] New subscription on topic '{topic}'")
    def publish(self, topic, message):
        with self.lock:
            queues = self.subscribers.get(topic, [])
                print(f"[Broker] Publishing on '{topic}': {message} to {len(queues)}
subscriber(s)")
        for q in queues:
            q.put((topic, message))
def subscriber(name, broker, topic):
    q = queue.Queue()
    broker.subscribe(topic, q)
    while True:
        topic, message = q.get()
        if message == "STOP":
            print(f"[Subscriber-{name}] Stopping.")
            break
        print(f"[Subscriber-{name}] Received on '{topic}': {message}")
def publisher(name, broker, topic, messages):
    for msg in messages:
        full_msg = f"{name} says: {msg}"
```

```
        broker.publish(topic, full_msg)
        time.sleep(random.uniform(0.5, 1.5))
    # Send STOP to indicate end of stream (optional)
    broker.publish(topic, "STOP")
if __name__ == "__main__":
    broker = Broker()
    # Subscriber threads
    sub1 = threading.Thread(target=subscriber, args=("A", broker, "sports"))
    sub2 = threading.Thread(target=subscriber, args=("B", broker, "news"))
    sub3 = threading.Thread(target=subscriber, args=("C", broker, "sports"))
    sub1.start()
    sub2.start()
    sub3.start()
    # Publisher threads
    pub1_messages = ["Team X won!", "Big match tomorrow.", "New player signed."]
    pub2_messages = ["Elections coming soon.", "New policy announced.", "Stock market rising."]
    pub1 = threading.Thread(target=publisher, args=("SportsPublisher", broker, "sports", pub1_messages))
    pub2 = threading.Thread(target=publisher, args=("NewsPublisher", broker, "news", pub2_messages))
    pub1.start()
    pub2.start()
    pub1.join()
    pub2.join()
    # Give some time for subscribers to consume STOP
    time.sleep(2)
    print("[Main] Ending program. Note: subscriber threads may still be running if STOP not handled for all topics.")
```

Execution:

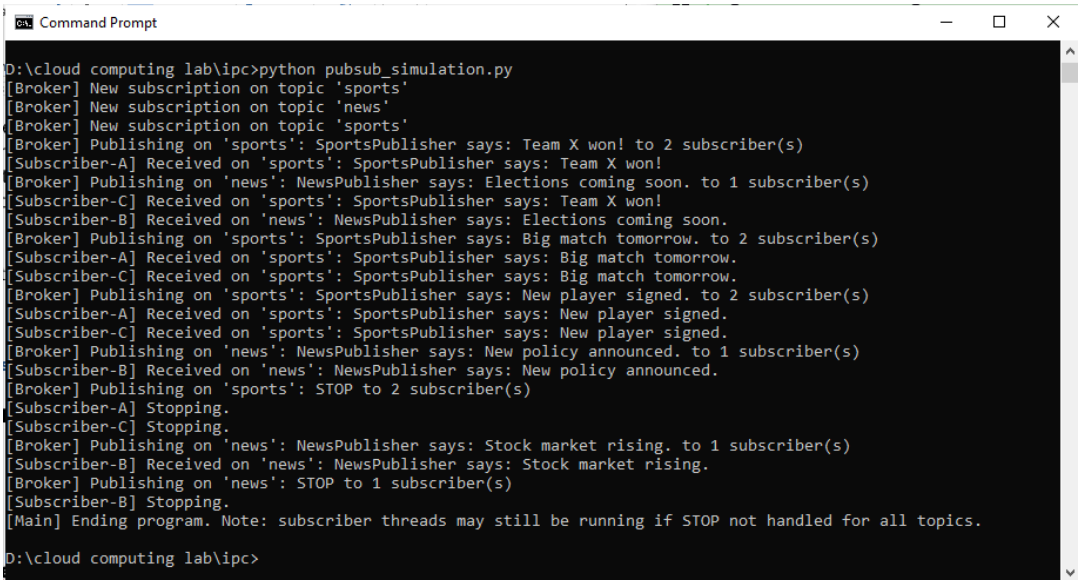
```
D:\cloud computing lab\ipc>python pubsub_simulation.py
[Broker] New subscription on topic 'sports'
[Broker] New subscription on topic 'news'
[Broker] New subscription on topic 'sports'
[Broker] Publishing on 'sports': SportsPublisher says: Team X won! to 2 subscriber(s)
[Subscriber-A] Received on 'sports': SportsPublisher says: Team X won!
[Broker] Publishing on 'news': NewsPublisher says: Elections coming soon. to 1 subscriber(s)
[Subscriber-C] Received on 'sports': SportsPublisher says: Team X won!
[Subscriber-B] Received on 'news': NewsPublisher says: Elections coming soon.
[Broker] Publishing on 'sports': SportsPublisher says: Big match tomorrow. to 2 subscriber(s)
[Subscriber-A] Received on 'sports': SportsPublisher says: Big match tomorrow.
[Subscriber-C] Received on 'sports': SportsPublisher says: Big match tomorrow.
[Broker] Publishing on 'sports': SportsPublisher says: New player signed. to 2 subscriber(s)
```

[Subscriber-A] Received on 'sports': SportsPublisher says: New player signed.
[Subscriber-C] Received on 'sports': SportsPublisher says: New player signed.
[Broker] Publishing on 'news': NewsPublisher says: New policy announced. to 1 subscriber(s)
[Subscriber-B] Received on 'news': NewsPublisher says: New policy announced.
[Broker] Publishing on 'sports': STOP to 2 subscriber(s)
[Subscriber-A] Stopping.
[Subscriber-C] Stopping.
[Broker] Publishing on 'news': NewsPublisher says: Stock market rising. to 1 subscriber(s)
[Subscriber-B] Received on 'news': NewsPublisher says: Stock market rising.
[Broker] Publishing on 'news': STOP to 1 subscriber(s)
[Subscriber-B] Stopping.
[Main] Ending program. Note: subscriber threads may still be running if STOP not handled for all topics.

D:\cloud computing lab\ipc>

Output Screens

Screenshot 1 — CMD showing executing pubsub_simulation.py



VIVA QUESTIONS

1. What is the Publish/Subscribe model?
2. What is a publisher?
3. What is a subscriber?
4. What is a topic?
5. Can a publisher know its subscribers? Why not?
6. What happens if a subscriber is offline during message broadcast?
7. Compare fanout, direct, and topic-based pub-sub models.
8. When would you use pub-sub instead of a queue?
9. What is message duplication in pub-sub systems?
10. Explain the importance of durable subscriptions.

11. How does pub-sub enable scalability in distributed systems?

EXERCISE 2.E — IPC USING RABBITMQ (PUBLISH/SUBSCRIBE MODEL)

AIM

To implement Distributed Inter-Process Communication using RabbitMQ with the Publish/Subscribe (Fanout Exchange) model in Python, demonstrating message broadcasting to multiple subscribers.

SOFTWARE / TOOLS REQUIRED

- Python 3.x
- Erlang Runtime
- RabbitMQ Server
- Python Library: pika
- Code Editor (VS Code / PyCharm / Notepad++)
- Command Prompt / Terminal

THEORY (Brief)

RabbitMQ is a message broker that enables communication between distributed applications.

Working Principle

1. Producer (Publisher) sends messages to an Exchange
2. Exchange routes messages to one or more Queues
3. Queue stores messages
4. Consumer (Subscriber) receives messages from the queue

Exchange Types

- Fanout – Broadcasts messages to all queues (used in this experiment)
- Direct – Routes using exact routing keys
- Topic – Pattern-based routing
- Headers – Header-based routing

INSTALLATION STEPS

Step 1: Install Erlang

Download and install from: https://www.erlang.org/downloads

Step 2: Install RabbitMQ

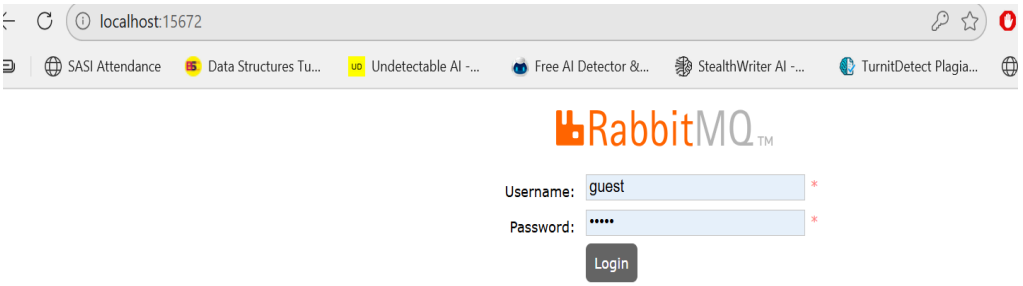
Download and install from: https://www.rabbitmq.com/download.html

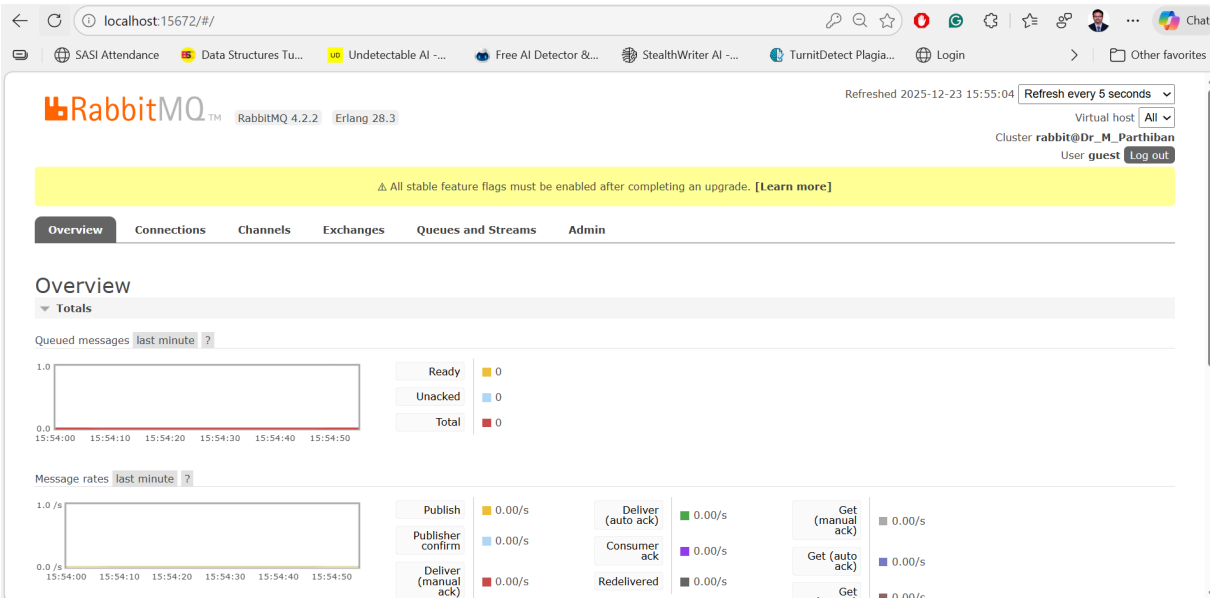
Step 3: Start RabbitMQ (Windows)

1. Press Windows + R
2. Type services.msc
3. Find RabbitMQ Server
4. Right-click → Start

Verify RabbitMQ
Open browser: http://localhost:15672
Login:

- **Username: guest**
- **Password: guest**





Step 4: Install Python Library
Open Command Prompt and run:
python -m pip install pika

EXPERIMENT PROCEDURE
Step 1: Create Project Folder
Create a folder: experiment2_IPC
Step 2: Create Subscriber Program
File Name: rabbitmq_subscriber.py

```
import pika
# Connect to RabbitMQ
connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
```

```
)
channel = connection.channel()
# Declare fanout exchange
channel.exchange_declare(exchange='logs', exchange_type='fanout')
# Create temporary queue
result = channel.queue_declare(queue='', exclusive=True)
queue_name = result.method.queue
# Bind queue to exchange
channel.queue_bind(exchange='logs', queue=queue_name)
print("[*] Waiting for messages. To exit press CTRL+C")
# Callback function
def callback(ch, method, properties, body):
    print("[Subscriber] Received:", body.decode())
channel.basic_consume(
    queue=queue_name,
    on_message_callback=callback,
    auto_ack=True
) channel.start_consuming()
```

Step 3: Create Publisher Program

File Name: rabbitmq_publisher.py

```
import pika
import time
# Connect to RabbitMQ
connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)
channel = connection.channel()
# Declare fanout exchange
channel.exchange_declare(exchange='logs', exchange_type='fanout')
messages = [
    "Hello Subscribers!",
    "This is a RabbitMQ Pub-Sub Demo",
    "Final Broadcast Message!"
]
for msg in messages:
    channel.basic_publish(
        exchange='logs',
        routing_key='',
        body=msg
    )
    print("[Publisher] Sent:", msg)
    time.sleep(1)
connection.close()
```

EXECUTION STEPS

Step 1: Run Subscribers

Open two separate Command Prompt windows.

In each window:

python rabbitmq_subscriber.py

Output:

[*] Waiting for messages. To exit press CTRL+C

Step 2: Run Publisher

Open a third Command Prompt:

python rabbitmq_publisher.py

EXPECTED OUTPUT

Publisher Output

[Publisher] Sent: Hello Subscribers!

[Publisher] Sent: This is a RabbitMQ Pub-Sub Demo

[Publisher] Sent: Final Broadcast Message!

Subscriber Output (Both Subscribers)

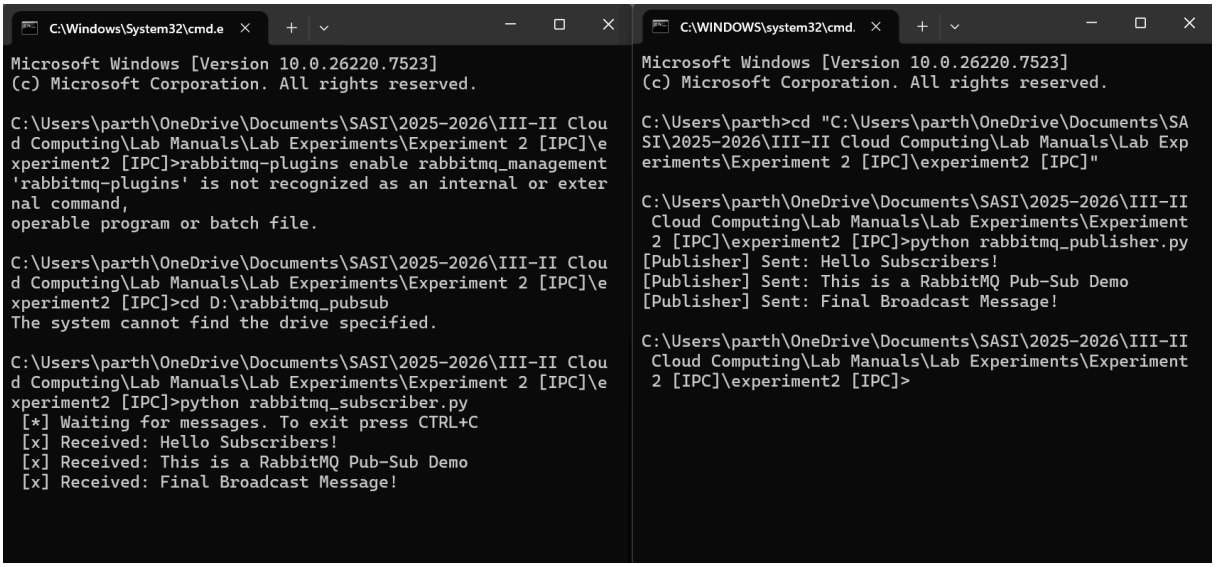
[Subscriber] Received: Hello Subscribers!

[Subscriber] Received: This is a RabbitMQ Pub-Sub Demo

[Subscriber] Received: Final Broadcast Message!

RESULT

Thus, IPC using RabbitMQ Publish/Subscribe model was successfully implemented and verified using Python.



VIVA QUESTIONS

1. What is RabbitMQ?
2. What is an exchange?
3. What is a queue?
4. What is the purpose of a fanout exchange?
5. How do publishers send messages to RabbitMQ?
6. How do subscribers receive messages?
7. Why does each subscriber get its own queue in pub-sub?
8. What is an acknowledgment (ack)?
9. What is message persistence?
10. How does RabbitMQ handle message routing?
11. Why is RabbitMQ more reliable than ZeroMQ for message delivery?
12. What happens when RabbitMQ crashes?
13. What is the difference between direct, fanout, and topic exchanges?
14. Explain how RabbitMQ supports load balancing in worker queues.

EXERCISE 2.F — IPC USING ZEROMQ (PUBLISH/SUBSCRIBE MODEL)

AIM

To implement Distributed Inter-Process Communication using ZeroMQ (ØMQ) by demonstrating the Publish/Subscribe messaging pattern for high-performance, broker-less message broadcasting.

SOFTWARE / TOOLS USED

- Python 3.x installed
- Python library: pyzmq
- A code editor (VS Code / PyCharm / Notepad++ / etc.)
- Terminal / Command Prompt

THEORY (Brief)

ZeroMQ (ØMQ) is a **high-performance messaging library** that enables communication between processes, applications, or systems over a network **without requiring a central message broker**.

In the **Publish/Subscribe (Pub-Sub)** model:

- A **Publisher** sends messages.
- One or more **Subscribers** receive messages.
- Publishers do **not know** who the subscribers are.
- Subscribers receive messages only while they are online.

INSTALLATION STEPS

Step 1: Install ZeroMQ Python Library

Open Command Prompt and run: python -m pip install pyzmq

EXPERIMENT PROCEDURE

Step 1: Create Project Folder

Create a folder named: experiment2_zeromq

Step 2: Create Subscriber Program

File Name: zeromq_subscriber.py

zeromq_subscriber.py

```
import zmq
context = zmq.Context()
socket = context.socket(zmq.SUB)
# connect to publisher
socket.connect("tcp://127.0.0.1:5555")
```

```
# subscribe to all messages
socket.setsockopt_string(zmq.SUBSCRIBE, "")
print("Subscriber ready...")
while True:
    message = socket.recv_string()
    print("Received:", message)
```

zeromq_subscriber.py

```
import zmq
import time
context = zmq.Context()
socket = context.socket(zmq.PUB)
# bind to a port so subscribers can connect
socket.bind("tcp://127.0.0.1:5555")
messages = ["Hello", "ZeroMQ Pub-Sub Example", "Bye"]
while True:
    for msg in messages:
        print("Sending:", msg)
        socket.send_string(msg)
        time.sleep(1)
```

Experiment Procedure

Step 1: Start two terminals for subscribers

Run zeromq_subscriber.py in both terminals.

Step 2: Start a third terminal for the publisher

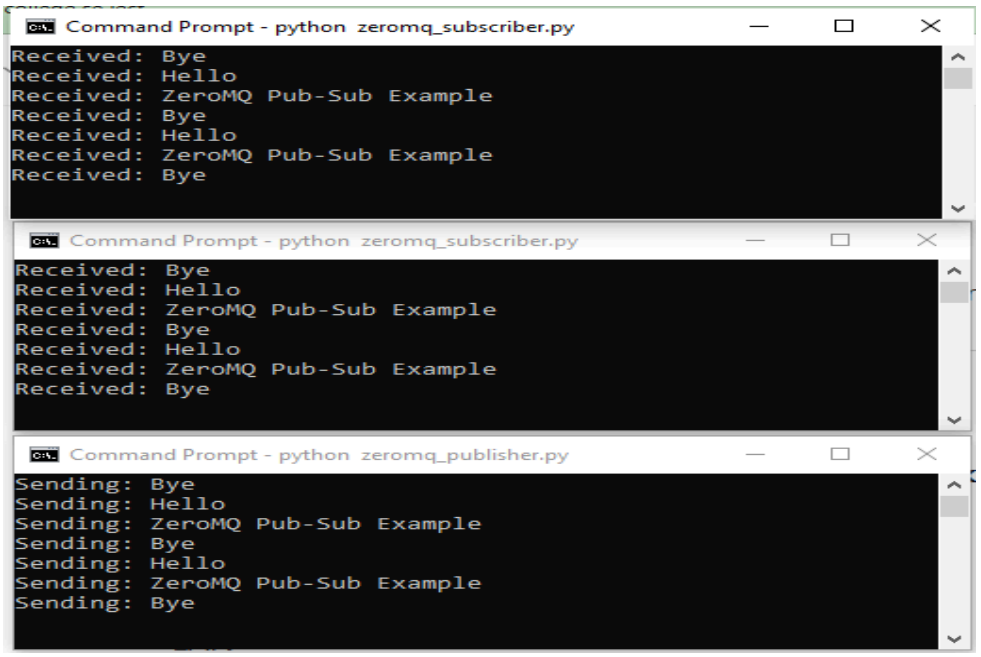
Run zeromq_publisher.py.

Step 3: Observe that every subscriber receives all messages broadcast by the publisher.

Output Screens

Screenshot 1 — CMDs showing executing

- *zeromq_subscriber.py*
- *zeromq_subscriber.py*
- *zeromq_publisher.py*



VIVA QUESTIONS

1. What is ZeroMQ?
2. Does ZeroMQ require a central server?
3. Which messaging patterns does ZeroMQ support?
4. How does ZeroMQ PUB–SUB differ from RabbitMQ PUB–SUB?
5. What are REQ/REP and PUSH/PULL patterns in ZeroMQ?
6. Why is ZeroMQ faster than RabbitMQ?
7. What happens to messages when a ZeroMQ subscriber is offline?
8. Why is ZeroMQ considered a “smart socket” library?
9. Compare ZeroMQ’s architecture with RabbitMQ’s broker-based system.